

22: Generative Modeling

Date: April 14, 2026

Acknowledgements. Some material and structure in these notes draw inspiration from the concise introductory course notes on deep generative models based on Stanford CS236, available at <https://deepgenerativemodels.github.io/notes/>.

1 Introduction

Generative modeling ask the following question: how can we (approximately) sample from an unknown probability distribution p over a space $\mathcal{X}$, given samples $x_1, \dots, x_n \sim p$? Last lecture we looked at the simpler problem of sampling with the knowledge of p . In this lecture, we will look at the full problem.

Modeling the true underlying data distribution p_{data} (from which our samples $x_1, \dots, x_n$ are drawn) is challenging, especially in high dimensions (*curse of dimensionality*). For example, the space of possible color images from a modern phone camera (approx. 700×1400 pixels) is astronomically large ($256^{3 \times 700 \times 1400} \approx 10^{800000}$), vastly exceeding even large datasets like ImageNet (around 15 million images). Fortunately, real-world data is highly structured (e.g., images are not random noise but contain objects, textures, etc.); generative models implicitly learn this underlying structure directly from the data, making the learning problem feasible. The standard approach is to define a parametric model $p_{\theta}(x)$, often using expressive function approximators like neural networks, and then learn the parameters θ such that $p_{\theta} \approx p_{\text{data}}$. (Recall Gaussian Mixture Models (GMMs), which we previously encountered for clustering, as an example where we explicitly modeled the density as a mixture and learned parameters via maximum likelihood).

There are broadly two strategies to achieve this: (1) explicitly model the density $p_{\theta}(x)$, learn θ (e.g., via maximum likelihood), and then sample from $p_{\theta}(x)$ using techniques like those from the previous lecture; or (2) learn a procedure that directly generates samples approximating those from p_{data} without explicitly forming $p_{\theta}(x)$. We will explore both, focusing primarily on the first.

Today, we examine several foundational models exemplifying these strategies: Variational Autoencoders (VAEs) and Autoregressive Models (explicit density), and Generative Adversarial Networks (GANs) (direct sampling). For each, we will consider its representation, learning objective, and optimization procedure. Future lectures will cover Diffusion Models and Transformers.

Why build these models? A well-learned p_{θ} enables key inference tasks:

- **Density estimation:** Evaluate $p_{\theta}(x)$ for any x .
- **Sampling:** Generate novel data $\tilde{x} \sim p_{\theta}(x)$.
- **Representation learning:** Obtain meaningful latent features z for x .

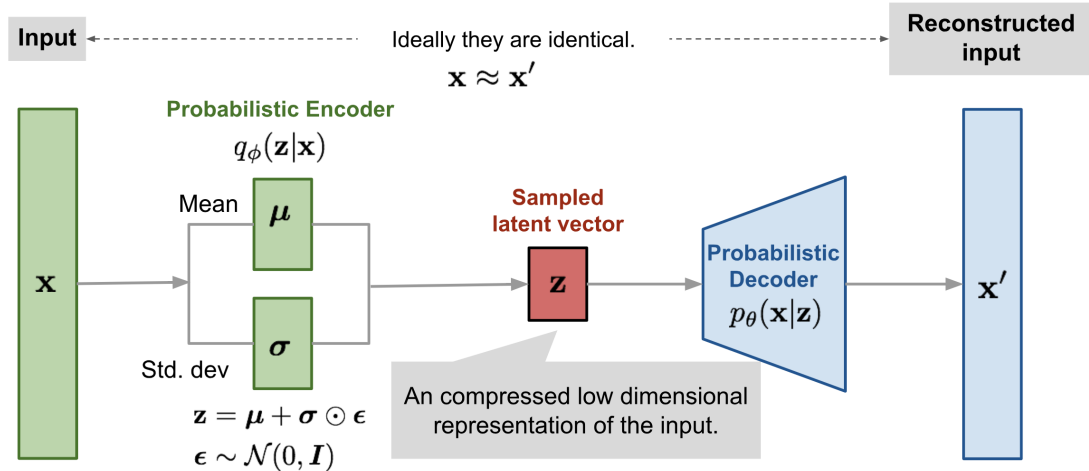


Figure 1: VAE as an auto-encoder. Source: Lilian Weng.

2 Latent Variable models

Often instead of modelling $p(x)$ directly, we model a joint probability distribution $p_\theta(x, z) = p_\theta(x|z)p(z)$ where z is some unknown latent variable. For example, in the case of modelling faces, we can think of latent variables as high level features such as gender, eye color, hair color, pose, etc. which are not observed. One could imagine that $p_\theta(x|z)$ is simple even when $p(x)$ is super complex.

Note that this is very similar to the mixture of Gaussians where the latent variable is the cluster center, and the mean and variance of each center are the parameters.

Autoencoders. Autoencoders are a method to learn low dimensional representations of inputs, sort of like a non-linear PCA. They consist of two components: an encoder that maps the input to a low dimensional latent representation, and a decoder that reconstructs the data from the latent representation. In particular, we solve the following optimization,

$$\arg \min_{E,D} \frac{1}{n} \sum_{i=1}^n \|x_i - D(E(x_i))\|^2$$

where E, D are usually neural networks. Now we can use $E(x)$ as a low dimensional projection of x that preserves *enough information* to reconstruct back x . However, standard AEs are not inherently generative; there's no guarantee that sampling a random point z in the latent space and decoding it ($D(z)$) will produce realistic output, as the latent space may not be structured appropriately. VAEs address this by introducing a probabilistic approach to the latent space.

2.1 Variational Autoencoders (VAEs)

Let us look at an instance of the latent variable model, Variational Autoencoders [Kingma and Welling, 2014] which builds on the encoder-decoder structure of autoencoders but introduces prob-

abilistic elements (see Figure 1). The overall goal remains to model the data distribution $p_\theta(x)$. In the latent variable approach, we achieve this indirectly by defining a simple prior distribution $p(z)$ over latent variables and a conditional distribution $p_\theta(x|z)$ (parameterized by the decoder network) that generates data x given z . The data distribution $p_\theta(x)$ is then defined implicitly as the marginal distribution obtained by integrating out the latent variable: $p_\theta(x) = \int p_\theta(x|z)p(z)dz$.

Learning Objective: The ELBO. As in GMM, we would like to maximize log-likelihood:

$$\log p_\theta(x) = \log \int p_\theta(x, z)dz = \log \int p_\theta(x|z)p(z)dz.$$

However, evaluating this integral is generally intractable when z is a high-dimensional continuous variable. The integral spans the entire latent space $\mathcal{Z}$ (e.g., $\mathbb{R}^{d_z}$), and computing high-dimensional integrals is computationally infeasible. Recall that in GMMs, the latent variable z represented the discrete cluster assignment (taking one of K values), turning the integral into a tractable sum over K components: $\sum_{k=1}^K p(z=k)p(x|z=k)$. Here, with continuous z , we cannot simply sum.

To make this tractable, we introduce a function $q_\phi(z|x)$ to approximate $p_\theta(z|x)$ and maximize the *Evidence lower bound* (ELBO) objective,

$$\log \int p_\theta(x, z)dz = \log \int p_\theta(x, z) \frac{q_\phi(z|x)}{q_\phi(z|x)} dz = \log E_{q_\phi} \left[\frac{p_\theta(x, z)}{q_\phi(z|x)} \right] \geq E_{q_\phi} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right].$$

Here the last inequality follows from Jensen’s inequality. Since the logarithm is a concave function, Jensen’s inequality states that $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$ for any random variable Y . In our case, $Y = \frac{p_\theta(x, z)}{q_\phi(z|x)}$ and the expectation is over $z \sim q_\phi(z|x)$.

In VAEs, we directly maximize this lower bound (the ELBO). The ELBO term can be further simplified to,

$$\begin{aligned} E_{q_\phi} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right] &= E_{q_\phi} \left[\log \frac{p_\theta(x|z)p(z)}{q_\phi(z|x)} \right] \\ &= E_{q_\phi} [\log p_\theta(x|z)] - E_{q_\phi} \left[\log \frac{p(z)}{q_\phi(z|x)} \right] \\ &= \underbrace{E_{q_\phi} [\log p_\theta(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{KL}(q_\phi(z|x) || p(z))}_{\text{prior matching term}}. \end{aligned} \quad (1)$$

Here KL divergence $D_{KL}(P||Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$ measures the distance between distributions P and Q . Note that it is asymmetric ($D_{KL}(P||Q) \neq D_{KL}(Q||P)$). An important consequence is that $D_{KL}(P||Q)$ becomes very large if $Q(x)$ is close to zero where $P(x)$ is non-zero, effectively forcing the model Q to cover the support of the true distribution P . Conversely, the penalty is small if $Q(x)$ is large where $P(x)$ is small.

ELBO Interpretation. The two terms in the above equation each have intuitive descriptions: the first term measures the reconstruction likelihood of the decoder from our variational distribution; this ensures that the learned distribution is modeling effective latents that the original data can be regenerated from. The second term measures how similar the learned variational distribution is to a prior belief held over latent variables. Minimizing this term encourages the encoder

to actually learn a distribution rather than collapse into a Dirac delta function. Maximizing the ELBO is thus equivalent to maximizing its first term and minimizing its second term.

Observe that $q_\phi(z|x)$ can be thought of as an *encoder* since it transforms data x to a latent representation z , and $p_\theta(x|z)$ can be thought of as a *decoder* that converts latent states to data. This encoder-decoder idea is essentially similar to autoencoders we described above. The latent space z serves as a compressed, often non-linear, representation of the input x , akin to the goals of dimensionality reduction techniques like PCA discussed previously, but learned through the probabilistic framework.

Another way to interpret this objective is that we would want to maximize log-likelihood of the data while ensuring $q_\phi(z|x)$ to be as close to $p_\theta(z|x)$ as possible.

Optional: ELBO and KL Divergence

The KL divergence between the approximate posterior $q_\phi(z|x)$ and the true posterior $p_\theta(z|x)$ relates directly to the ELBO:

$$\begin{aligned} D_{KL}(q_\phi(z|x)||p_\theta(z|x)) &= \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)}{p_\theta(z|x)} \right) dz \\ &= \int q_\phi(z|x) \left(\log(p_\theta(x)) + \log \left(\frac{q_\phi(z|x)}{p_\theta(x,z)} \right) \right) dz \\ &= \log p_\theta(x) + \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)}{p_\theta(x,z)p(z)} \right) dz \\ &= \log p_\theta(x) + \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)}{p(z)} \right) - \log(p_\theta(x|z)) dz \\ &= \log p_\theta(x) + \underbrace{D_{KL}(q_\phi(z|x), p(z)) - E_{q_\phi}[\log p_\theta(x|z)]}_{-\text{ELBO}}. \end{aligned}$$

Rearranging, we get

$$\text{ELBO} = \log p_\theta(x) - D_{KL}(q_\phi(z|x)||p_\theta(z|x))$$

which shows maximizing the ELBO is equivalent to minimizing the KL divergence between the approximate and true posterior, while maximizing the log-likelihood of the data.

Implementation with Gaussian Distributions. In VAE, we take $p(z)$ to be the standard normal distribution $\mathcal{N}(0, I)$, and parameterize the approximate posterior (encoder) as $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)I)$ and the likelihood (decoder) as $p_\theta(x|z) = \mathcal{N}(\mu_\theta(z), \Sigma_\theta(z))$. The encoder network, parameterized by ϕ , takes x as input and outputs the parameters $\mu_\phi(x)$ and $\sigma_\phi(x)$ of the Gaussian distribution representing the approximate posterior for z . Now the KL term in the ELBO (Eq. 1) can be computed analytically (for Gaussians), and the reconstruction term $E_{q_\phi}[\log p_\theta(x|z)]$ can be approximated using a Monte-Carlo estimate (from last lecture). Recall that to perform a Monte-Carlo estimate of the reconstruction term, we first sample $z_1, \dots, z_L \sim q_\phi(z|x)$ for data point x

(using the reparameterization trick described below), and then compute

$$E_{q_\phi} [\log p_\theta(x|z)] \approx \frac{1}{L} \sum_{l=1}^L \log p_\theta(x|z_l)$$

Training and the Reparameterization Trick. There is one last challenge. Since z_l are all sampled from q_ϕ , it is not clear how to compute gradients with respect to ϕ . There is a clever *reparameterization* trick that helps us address this. The reparameterization trick rewrites a random variable as a deterministic function of its parameters and an independent noise variable. For example, samples from a normal distribution $z \sim \mathcal{N}(\mu, \sigma^2)$ can be rewritten as:

$$z = \mu + \sigma\epsilon \text{ where } \epsilon \sim \mathcal{N}(0, 1).$$

Therefore, sampling from $q_\phi(z|x)$ is equivalent to sampling noise variable $\epsilon \sim \mathcal{N}(0, 1)$ and computing $z = \mu_\phi(x) + \sigma_\phi(x)\epsilon$. Since the sampling of ϵ does not depend on ϕ , we can now backpropagate through the deterministic computation of z to optimize ϕ . Now we can optimize the ELBO over both ϕ and θ using gradient ascent.

Why not directly estimate $\log p_\theta(x)$ with Monte Carlo? One might wonder why we don't just directly estimate the intractable integral $p_\theta(x) = \int p_\theta(x|z)p(z)dz = E_{z \sim p(z)}[p_\theta(x|z)]$ using Monte-Carlo sampling from the prior $p(z)$. That is, sample $z^{(l)} \sim p(z)$ for $l = 1..L$ and approximate $p_\theta(x) \approx \frac{1}{L} \sum_l p_\theta(x|z^{(l)})$, then maximize the log of this estimate.

The main issue is *high variance*, especially in high dimensions. For a specific data point x , most latent codes z sampled from the broad prior $p(z)$ will produce reconstructions $p_\theta(x|z^{(l)})$ with very low likelihood (close to zero). Only a small region of the latent space corresponds well to x . Therefore, the Monte Carlo estimate $\hat{p}_\theta(x)$ will be dominated by noise unless L is astronomically large, making the gradient estimates for θ very unreliable during training. The ELBO approach avoids this by using the *encoder* $q_\phi(z|x)$ to sample z for the reconstruction term ($E_{q_\phi(z|x)}[\log p_\theta(x|z)]$). Since $q_\phi(z|x)$ is trained to propose z 's relevant to the specific x , this sampling is much more focused, leading to a lower-variance and more practical objective for optimization.

Generating New Samples. To produce new data, we can throw out the encoder, sample $z \sim \mathcal{N}(0, I)$ from the prior, and then sample x using the decoder $p_\theta(x|z)$. Even though the basic VAE formulation described here often produces blurry images (due to factors like the reconstruction loss), more advanced variants have improved sharpness. However, we will look at diffusion models next class, which currently represent a significant leap in generation quality for tasks like image synthesis.

3 Autoregressive models

Having explored latent variable models like VAEs, we now turn to a different approach for modeling sequential data: autoregressive models. Generally for these models we assume the data is a sequence

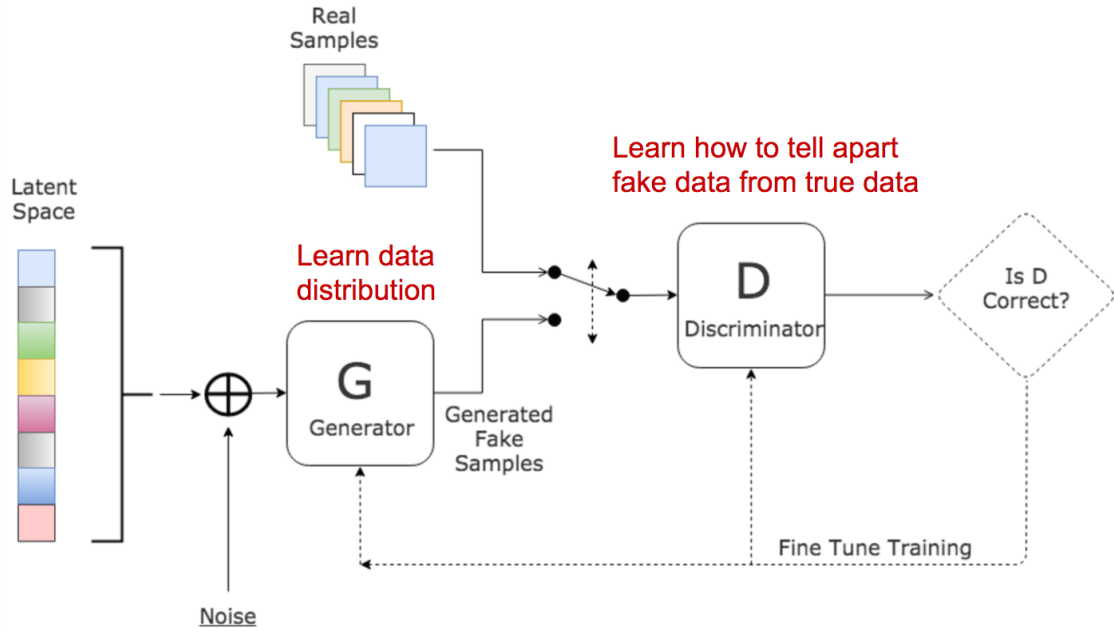


Figure 2: Illustration of GAN. Source: www.kdnuggets.com/2017/01/generative-...-learning.html.

$(x_1, \dots, x_T)$, and we can decompose the likelihood by Bayes Rule as,

$$p(x_1, \dots, x_T) = \prod_{t=1}^t p(x_t | x_1, \dots, x_{t-1}) = \prod_{t=1}^t p(x_t | x_{<t}).$$

So the maximum likelihood objective decomposes as a sequence of prediction problems for each term in the sequence given the previous terms.

$$\log p(x_1, \dots, x_T) = \sum_{t=1}^t \log p(x_t | x_{<t}).$$

As usual, we will parameterize the conditionals using neural networks, and train them using MLE. In two lectures from now, we will see how this autoregressive models serves as the base for large language models.

Now observe that, to sample from this autoregressive models, we can follow the sequential process: first sample x_1 , then sample x_2 conditioned on x_1 and so on.

4 Generative Adversarial Networks (GANs)

Instead of modeling $p(x)$ which can be challenging, can we circumvent this to directly generate samples? GANs [Goodfellow et al., 2014] achieve this in the following way:

- Introduce a latent variable with prior $p(z)$
- Sample $z \sim p(z)$ and pass it through a generator network G to get $x = G(z)$.

This provides a way to *directly* sample approximate data points x by transforming samples from a simple prior distribution $p(z)$, without needing to explicitly model the complex data distribution $p(x)$ itself, unlike VAEs.

We want $p_G(z) \approx p(x)$, so how do we ensure that? We train G to fool a discriminator D that sees the "fake" sample generated by G and a true sample from the distribution. The discriminator is a classifier that decides "real" (label 1) or "fake" (label 0). This leads to the following min-max game:

$$\min_G \max_D \underbrace{E_{x \sim p_{\text{data}}} [\log D(x)]}_{\text{discriminator wants } D(x) = 1 \text{ for real data}} + \underbrace{E_{z \sim p(z)} [\log(1 - D(G(z)))]}_{\text{discriminator wants } D(x) = 0 \text{ for fake data}}$$

Now we can train this using gradient descent/ascent on G and D respectively. This turns out to not be super stable, and often leads to mode collapse where the generator only produces inputs of a few types instead of the full distribution. This instability relates to the broader challenges in sampling from high-dimensional or complex distributions, as touched upon in the previous lecture.